

Control de Capas del Mapa - GEO ARTE CDMX

Version: 1.0 Fecha: junio 2026 Audiencia: desarrolladores, administradores y usuarios avanzados Modulo: Mapa interactivo ('/mapa')

Tabla de contenidos

- 1. [Vision general](#1-vision-general)
- 2. [Arquitectura del flujo de datos](#2-arquitectura-del-flujo-de-datos)
- 3. [Catalogo de capas](#3-catalogo-de-capas)
- 4. [Estado en React](#4-estado-en-react)
- 5. [Panel UI (MapCapasPanel)](#5-panel-ui-mapcapaspanel)
- 6. [Comportamiento al marcar/desmarcar](#6-comportamiento-al-marcar-desmarcar)
- 7. [Presets y reset](#7-presets-y-reset)
- 8. [Fuentes de datos](#8-fuentes-de-datos)
- 9. [Archivos clave del codigo](#9-archivos-clave-del-codigo)
- 10. [Resumen ejecutivo](#10-resumen-ejecutivo)

1. Vision general

El Control de Capas del mapa de GEO ARTE CDMX no es un plugin de Leaflet. Es un panel React personalizado ('MapCapasPanel') que administra dos tipos de capas distintas:

Tipo	Que representa	Como se dibuja en el mapa
Infraestructura cultural	(12 tipologias SIC)	Puntos: bibliotecas, museos, teatros, etc. Marcadores circulares ('CircleMarker') agrupados por tipo
Capas territoriales / analisis	Poligonos y lineas: transporte, densidad, brechas, cobertura	GeoJSON sobre Leaflet ('L.geoJSON')

La configuracion de que capas existen, sus iconos, etiquetas y valores por defecto vive en un catalogo estatico ('mapaCapasSecciones'). El estado de que esta activo vive en React y se propaga al motor del mapa ('MapCanvas').

Importante: marcar o desmarcar un checkbox no dispara una nueva peticion al servidor. Los datos (espacios culturales, poligonos, metricas) ya estan cargados al abrir el mapa; el panel solo decide que mostrar u ocultar.

2. Arquitectura del flujo de datos

```
/mapa
%% % MapaLoader (Suspense + dynamic import, sin SSR)
%% % % MapaController
%% % % % GET /api/data/mapa
%% % % % % getMapaData()
%% % % % % Espacios culturales (Supabase o mock)
%% % % % % Datos territoriales (poligonos, metricas, transporte)
%% % % % % Configuracion UI (mapaCapasSecciones, presets, colores)
%% % % MapaInteractivo
%% % % % MapCapasPanel !• panel lateral de capas
%% % % % MapCanvas !• motor Leaflet
%% % % % % map-overlays.ts !• capas GeoJSON
```

Pasos del flujo

- 1. La ruta /mapa carga MapaLoader, que importa MapaController de forma dinamica (ssr: false) porque Leaflet requiere el DOM del navegador.
- 2. MapaController solicita datos a /api/data/mapa, que ejecuta getMapaData() en el servidor.

3. Esa funcion obtiene:

- Espacios culturales con coordenadas (desde Supabase o datos mock de demostracion). - Datos territoriales: poligonos de alcaldias y macrozonas, metricas de cobertura/brecha, red de transporte. - Configuracion estatica del panel: secciones, presets de vista, colores por tipologia.

1. Con los datos cargados, MapaInteractivo renderiza el panel y el mapa. Los cambios en los checkboxes actualizan el estado local y, en cascada, lo que Leaflet muestra.

3. Catalogo de capas

El catalogo esta definido en 'src/lib/data/mock/mapa.ts' como 'mapaCapasSecciones'. Cada capa tiene esta estructura:

```
{
  id: "bibliotecas",
  label: "Bibliotecas",
  subtitulo: null,
  icon: "library",
  mapKey: "bibliotecas",    // null para capas territoriales
  defaultChecked: false,
  defaultOpacity: 80,
  conOpacidad: true,
}
```

Secciones del panel

Seccion | ID | Contenido
Infraestructura Cultural | 'infra' | 12 tipologias SIC con 'mapKey'
Variables Territoriales | 'territorial' | Transporte, densidad, vacios territoriales
Analisis Geo Arte | 'analisis' | Cobertura cultural, recursos cualitativos

Las 12 tipologias SIC (infraestructura)

mapKey | Etiqueta en UI
'auditorios' | Auditorios
'bibliotecas' | Bibliotecas
'bibliotecasDgb' | Bibliotecas DGB
'casasArtesanias' | Casas de artesanias
'casasCentrosCulturales' | Casas y centros culturales
'centrosPueblosIndigenas' | Centros Coord. de pueblos indigenas
'complejosCinematograficos' | Complejos cinematograficos
'galerias' | Galerias
'libreriasPuntosVenta' | Librerias y puntos de venta
'museos' | Museos
'teatros' | Teatros
'universidades' | Universidades

Capas sin 'mapKey' (overlays y analisis)

ID | Etiqueta | Tipo de geometria
'transporte' | Transporte Masivo | Lineas (Metro, Metrobus, Cablebus)
'densidad' | Densidad por macrozona | Poligonos coloreados
'nivel' | Vacios territoriales | Poligonos de alcaldias con brecha alta
'cobertura' | Cobertura Cultural | Poligonos de alcaldias por % cobertura
'recursosCualitativos' | Recursos cualitativos | Puntos (entrevistas/encuestas)

Regla de enlace

- Con mapKey -> capa de puntos (infraestructura SIC). El mapKey coincide con EspacioTipo en src/lib/domain/mapa.ts.
- Con mapKey -> capa de puntos (infraestructura SIC). El mapKey coincide con EspacioTipo en src/lib/domain/mapa.ts.

4. Estado en React

El componente central 'MapaInteractivo' mantiene dos objetos de estado principales.

4.1 'capaMapa' - las 12 tipologías SIC

Tipo definido en 'src/lib/domain/mapa.ts':

```
type CapaMapaState = Record<EspacioTipo, { visible: boolean; opacity: number }>;
```

Ejemplo conceptual:

```
{
  bibliotecas: { visible: true, opacity: 80 },
  museos: { visible: false, opacity: 80 },
  // ... resto de tipologías
}
```

Inicialización: la función 'buildCapaMapaInitial()' recorre 'mapaCapasSecciones' y asigna 'defaultChecked' y 'defaultOpacity' de cada capa con 'mapKey'.

4.2 'capasExtra' - overlays y recursos cualitativos

Tipo en 'src/lib/domain/mapa-territorial.ts':

```
type MapaCapasToggleId =
  | "transporte"
  | "densidad"
  | "nivel"
  | "cobertura"
  | "recursosCualitativos";
```

Cada ID es un booleano ('true' = visible en el mapa).

Inicialización: 'buildCapasExtraInitial()' lee 'defaultChecked' de las capas sin 'mapKey'.

4.3 Sincronización con filtros avanzados

Al cambiar la visibilidad de una tipología desde el panel, 'onCapaMapaChange' también actualiza 'filtrosAplicados.tipos' para mantener coherencia entre el panel de capas y el panel de Filtros avanzados.

5. Panel UI (MapCapasPanel)

Archivo: 'src/components/mapa/MapCapasPanel.tsx'

Es un componente presentacional: no conoce Leaflet. Solo recibe estado y callbacks.

Elementos de interfaz

Elemento | Función

Cabecera con título | Muestra «Control de Capas»; botón para colapsar el panel

Secciones colapsables | Agrupan capas por categoría (Infraestructura, Territorial, Análisis)

'LayerItem' | Icono + nombre + checkbox (+ slider de opacidad si aplica)

Pie del panel | Fuente territorial, capa base activa (OpenStreetMap), botón «Resetear Vista»

Logica de enlace por capa

```
if (capa.mapKey !== null) {  
  // Usa capaMapa[capa.mapKey] -> onCapaMapaChange  
} else {  
  // Usa capasExtra[capa.id] -> onCapaExtraChange  
}
```

Opacidad

Solo las capas de infraestructura con 'conOpacidad: true' muestran un slider (rango 15-100 %) cuando estan activas. La opacidad se aplica a los marcadores del 'LayerGroup' correspondiente en Leaflet.

Colapso del panel

En pantallas grandes ('lg:'), el panel puede contraerse a una barra estrecha ('lg:w-12') para dar mas espacio al mapa.

6. Comportamiento al marcar/desmarcar

6.1 Capas de infraestructura (puntos culturales)

Secuencia:

1. El usuario marca «Bibliotecas» en el panel.
2. MapCapasPanel llama onCapaMapaChange("bibliotecas", { visible: true }).
3. MapaInteractivo actualiza capaMapa y sincroniza filtros avanzados.
4. espaciosFiltrados (useMemo) excluye espacios cuyo tipo no esta visible:

```
""ts base.filter((espacio) => capaMapa[espacio.tipo].visible) ""
```

1. MapCanvas recibe los espacios filtrados y:

- Crea/actualiza marcadores en 'syncEspacioMarkers()' - un 'CircleMarker' por espacio, con color segun 'tipoColors'. - Muestra u oculta el LayerGroup del tipo en un 'useEffect' sobre 'capaMapa': ""ts if (estado.visible) { group.addTo(map); applyOpacity(group, estado.opacity / 100); } else { group.remove(); } ""

Cada tipologia tiene su propio 'L.LayerGroup' en Leaflet. Los marcadores se agrupan por tipo para poder controlar visibilidad y opacidad de forma independiente.

6.2 Capas territoriales (GeoJSON)

Secuencia:

1. El usuario marca «Transporte Masivo».
2. onCapaExtraChange("transporte", true) actualiza capasExtra.
3. MapCanvas ejecuta syncOverlayVisibility() para anadir o quitar la capa del mapa.

Las geometrias se construyen en 'src/components/mapa/map-overlays.ts':

ID | Funcion constructora | Que dibuja
'transporte' | 'buildTransporteLayer()' | Lineas de Metro / Metrobus / Cablebus
'densidad' | 'buildDensidadLayer()' | Poligonos de macrozonas coloreados por densidad de infraestructura
'nivel' | 'buildVaciosLayer()' | Alcaldias con brecha territorial alta (vacios de oferta)
'cobertura' | 'buildCoberturaLayer()' | Alcaldias coloreadas por porcentaje de cobertura cultural

Cuando cambian los datos territoriales o los filtros de brecha ('brechaMinima', 'soloVacios'), 'rebuildOverlayLayers()' regenera las geometrias y vuelve a aplicar la visibilidad.

6.3 Recursos cualitativos (caso especial)

No pasa por 'map-overlays.ts'. Si 'capasExtra.recursosCualitativos === true', 'MapaInteractivo' pasa el arreglo 'recursosFiltrados'

a 'MapCanvas', que crea marcadores rosas ('#ec4899') independientes de los espacios del padron SIC.

6.4 Leyenda de capas activas

Cuando hay overlays territoriales visibles, 'MapCanvas' muestra una leyenda en la esquina inferior izquierda con los nombres de las capas activas ('OVERLAY_LEGEND').

7. Presets y reset

Selector de vista (barra superior)

Definido en 'mapaCapasPresets' y aplicado por 'applyMapaPreset()' en 'src/lib/mapa/map-presets.ts':

Preset	Valor	Efecto
Puntos culturales	'infra'	Activa las 12 tipologias SIC; desactiva todos los overlays
Analisis territorial	'territorial'	Oculto todos los puntos; activa densidad + cobertura + vacios

Boton «Resetear Vista»

1. Restaura capaMapa y capasExtra a los valores defaultChecked del catalogo.
2. Restaura filtros avanzados al estado por defecto.
3. Incrementa resetNonce, lo que hace que MapCanvas reencuadre el mapa en las 16 alcaldias de CDMX.

8. Fuentes de datos

Dato	Origen	Funcion / repositorio
Espacios (puntos SIC)	Supabase (o mock)	'fetchEspaciosCulturalesForMapa()'
Recursos cualitativos	Supabase (o mock)	'fetchRecursosCualitativosActivosFromSupabase()'
Poligonos de alcaldias / macrozonas	Supabase / GeoJSON / fallback	'fetchMapaTerritorialDataServer()'
Metricas de cobertura y brecha	Supabase / estimacion local	'fetchMapaTerritorialData()'
Red de transporte	Supabase / GeoJSON	'buildTransporteLayer()'
Catalogo del panel (nombres, iconos)	Estatico	'src/lib/data/mock/mapa.ts'
Colores por tipologia	Estatico	'ESPEACIO_TIPO_COLORS' en 'src/lib/domain/mapa.ts'
Capa base del mapa	OpenStreetMap tiles	'L.tileLayer' en 'MapCanvas'

Modos de operacion

- Supabase configurado y con datos: badge «Padron Supabase»; conteo real de espacios en dataSourceNote.
- Supabase configurado y con datos: badge «Padron Supabase»; conteo real de espacios en dataSourceNote.
- Supabase configurado y con datos: badge «Padron Supabase»; conteo real de espacios en dataSourceNote.

El pie del panel muestra un resumen de fuentes territoriales mediante 'formatTerritorialFuenteLabel()' (que capas vienen de Supabase vs. estimacion local).

9. Archivos clave del codigo

Archivo	Responsabilidad
'src/app/mapa/page.tsx'	Punto de entrada de la ruta '/mapa'
'src/components/mapa/MapaLoader.tsx'	Carga dinamica sin SSR
'src/components/features/mapa/MapaController.tsx'	Fetch de datos y estado de carga
'src/lib/services/mapa.service.ts'	'getMapaData()' - orquestacion servidor
'src/lib/data/mock/mapa.ts'	Catalogo 'mapaCapasSecciones', presets, colores

'src/lib/domain/mapa.ts' | Tipologías SIC, 'CapaMapaState', colores
'src/lib/domain/mapa-territorial.ts' | IDs de overlays, filtros territoriales
'src/components/mapa/MapaInteractivo.tsx' | Estado global, filtros, orquestación UI
'src/components/mapa/MapCapasPanel.tsx' | Panel lateral con checkboxes
'src/components/mapa/MapCanvas.tsx' | Motor Leaflet (marcadores + capas)
'src/components/mapa/map-overlays.ts' | Construcción de capas GeoJSON
'src/lib/mapa/map-presets.ts' | Presets «Puntos culturales» / «Análisis territorial»
'src/lib/mapa/filter-espacios.ts' | Filtrado por búsqueda, alcaldía y métricas

10. Resumen ejecutivo

El Control de Capas de GEO ARTE CDMX funciona así:

1. Un catálogo declarativo (mapaCapasSecciones) define que capas existen, como se llaman y sus valores por defecto.
2. Dos estados React (capaMapa para puntos SIC, capasExtra para overlays) registran que está activo.
3. El panel (MapCapasPanel) es UI pura: checkboxes y sliders que llaman callbacks.
4. MapaInteractivo filtra los datos según el estado y los pasa al mapa.
5. MapCanvas (Leaflet) muestra u oculta grupos de marcadores y capas GeoJSON según ese estado.
6. No hay peticiones nuevas al servidor al togglear capas; todo opera sobre datos ya cargados.

En una frase: el panel es la interfaz, React es el cerebro, Leaflet es el renderizado, y Supabase (o mock) es la fuente de datos inicial.

Documento técnico generado para el proyecto GEO ARTE Web. Para el manual de uso orientado al usuario final, consulte 'docs/cliente/MANUAL-USUARIO.md'.